Assignment V: Goal-Based Portfolio Optimisation

Objective

This assignment introduces goal-based investing. Your core objective is to design a static portfolio that maximizes the probability of achieving a final retirement goal. You will use a brute-force combination method on discrete portfolio weights and Monte Carlo simulations to find the optimal allocation.

Problem Setup & Financial Constraints

You are designing an investment plan for a client over an exact **20-year time horizon**.

- **Initial Savings:** The client starts with a savings rate of ₹20,000 per month.
- **Savings Growth:** The savings amount will increase at a rate of **4% annually**.
- **Borrowing Penalty:** If the simulated portfolio value falls short of an intermediate goal's target amount in the year it is due, you must borrow the shortfall amount at an exact **12% annual interest rate** to fulfill the goal. This loan must be paid off using future portfolio returns and savings.

Data Collection & Security Universe

You must build your portfolio using the following **5 uncorrelated Indian stocks**.

- **TCS.NS** (IT)
- **HDFCBANK.NS** (Financials)
- **RELIANCE.NS** (Energy/Conglomerate)
- **SUNPHARMA.NS** (Healthcare)
- **ITC.NS** (FMCG)

Data Requirements:

- Download daily adjusted closing prices for these 5 tickers using the yfinance Python library for the exact period of **January 1, 2014, to December 31, 2023** (the last 10 years).
- Calculate the annualized expected return, annualized risk (standard deviation), and the covariance matrix yourself using this historical data. Do not use external or assumed risk/return metrics.

Part 1: Goal Sequences

To demonstrate how different objectives alter the optimal portfolio composition , you will evaluate two completely different sequences of goals. Each sequence contains exactly three intermediate goals and one terminal retirement goal.

Sequence A (Aggressive Early Goals):

- Goal 1: ₹15 Lakhs at Year 3
- Goal 2: ₹25 Lakhs at Year 7
- Goal 3: ₹30 Lakhs at Year 12
- Terminal Goal (Retirement): ₹1.5 Crores at Year 20

Sequence B (Backloaded Goals):

- Goal 1: ₹10 Lakhs at Year 8
- Goal 2: ₹20 Lakhs at Year 12
- Goal 3: ₹40 Lakhs at Year 16
- Terminal Goal (Retirement): ₹1.5 Crores at Year 20

Part 2: Portfolio Simulation & Optimization

You must create a **static portfolio** that remains unchanged throughout the 20-year time horizon.

Task 2.1: Discrete Weight Allocation

- Generate every possible portfolio combination using only the following discrete weights for the 5 securities: **0, 0.25, 0.5, 0.75, and 1.0**.
- The sum of the weights for each valid combination must equal exactly 1.0.

Task 2.2: Monte Carlo Simulation

- For every valid discrete weight combination, run a Monte Carlo simulation with exactly **5,000 paths**.
- Apply the annual 4% savings increase and deduct goal amounts at their respective years.
- Apply the 12% borrowing logic if a path falls short of an intermediate goal.

Task 2.3: Maximizing Success

- Calculate the probability of success for the terminal retirement goal for each portfolio combination.
- Brute-force the results to identify the single portfolio combination that yields the highest probability of reaching the ₹1.5 Crore terminal goal.
- Run this optimization separately for Sequence A and Sequence B.

Deliverables

- **Python Code (.py or .ipynb):** Must include yfinance data extraction, statistical calculations, brute-force combination generation, and the Monte Carlo simulation.
- **Report (1-2 pages):** State the optimal portfolio weights for Sequence A and Sequence B. Briefly explain why the completely different sequences of goals resulted in different optimal portfolio compositions.

- Note: Real life might require dynamic portfolios, but this exercise strictly requires a static portfolio setup.

Bonus (Optional)

You can earn bonus points by modifying your code to bypass the discrete weights and brute-force method.

- Make the security weights completely continuous (any float between 0 and 1).
- Allow for short-selling (e.g., -0.5 in one security, 1.5 in another), ensuring the total sum of weights still equals exactly 1.0.
- Explicitly use an algorithmic optimizer (like SciPy's `minimize`) to find the optimal continuous weights.

Grading Rubric

Criteria	**Weight**	**Expectation**
Methodologies & Implementation	30%	A neat flowchart and clean documented code
Results and Presentation(Tasks)	50%	Good results with meaningful presentation
Results discussion & Bonus	20%	Discussion of the results in own words